Class 16: Feature Engineering — From Raw Data to Validated Features

Data-driven Systems Engineering

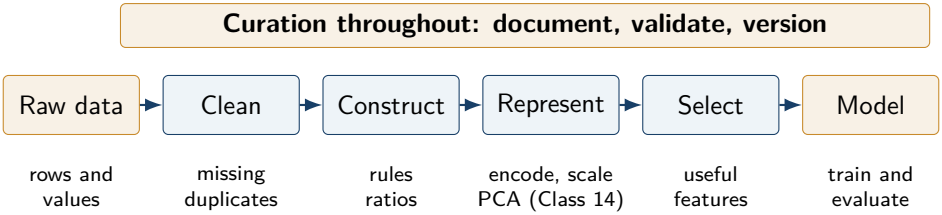
Agenda

- Why feature engineering remains necessary once every variable is numerically encoded.
- Data curation throughout the workflow: documentation, versioning, and validation.
- Data cleaning: missing values, duplicates, invalid values, and outliers.
- Feature construction from domain knowledge, for tabular data and data streams.
- Feature representation before selection: encoding, scaling, and PCA (Class 14).
- Feature selection: filter, wrapper, and embedded methods.
- Leakage and availability checks before deployment.
- Titanic running case: baseline, ablation, evaluation, and reproducible pipeline.

Why feature engineering remains necessary after encoding

- Class 14 explained numerical representations; this class places them in a workflow. Encoding alone does not make a variable correct, reliable, or informative.
- Real datasets contain missing values, duplicates, and inconsistent types, which must be resolved before an encoding is meaningful.
- Not every available variable should become a feature: some are redundant, some carry only noise, and some induce leakage.
- Informative features are frequently absent from the raw data and must be constructed from domain knowledge.

Feature engineering in the modeling workflow



Feature construction and transformation

Construction creates candidate features (e.g., ratios or rolling means). It is one kind of feature transformation; encoding and scaling also transform features, while selection chooses which candidates to keep.

The workflow is iterative: reject leaking or unavailable features early; construction and selection may be revisited. PCA constructs components rather than selecting original columns.

Running case: Titanic survival prediction

Engineering question

Can we predict whether a passenger survives, using only information available at the intended prediction moment?

- **Target:** `survived` (binary), $n = 891$ passengers.
- **Raw candidates:** `age`, `sex`, `pclass`, `fare`, `sibsp`, `parch`, `embarked`.
- **Suspicious candidates:** `alive` and any attribute defined using outcomes after the event.

Evidence we will reveal

Raw → Curate → Clean → Construct → Represent → Select → Validate.

Class discussion: Is a variable useful just because it is predictive?

Data curation

- Curation makes a dataset understandable and reusable beyond its original authors.
- A data dictionary records the meaning, unit, and valid range of every column.
- Capture provenance and lineage: source, extraction/query, timestamp, version, and feature definitions; an extraction date alone does not guarantee reproducibility.
- Validation rules (schema and range checks) detect upstream problems before they reach the modeling stage.

Repository example

For Titanic, record a feature contract: definition, type, allowed values, missingness policy, source, prediction-time availability, and version. The fraud-case specification in `documents/FraudDetectionAsaService.pdf` illustrates similar documentation.

Titanic / 1: curate the feature contract

Field	Type	Meaning / validation	Prediction-time audit
`survived`	binary	Target; 0 or 1	Never an input
`sex`	nominal	Passenger category	Check available
`age`	numeric	Age in years; missing possible	Check available
`sibsp`	count	Siblings/spouses aboard; ≥ 0	Check available
`parch`	count	Parents/children aboard; ≥ 0	Check available
`embarked`	nominal	Embarkation port; some missing	Check available
`alive`	nominal	Outcome-derived survival label	Exclude: target leakage

What should be versioned?

Dataset snapshot / extraction, column schema, validation rules, cleaning and construction code, split definition, and fitted preprocessing artifacts.

Question: Which fields would you reject *before* looking at feature scores?

Data cleaning: scope and quality dimensions

Profile, validate, and document the data

Missingness
Measure rates and patterns; check whether absence is meaningful. Choose to preserve, impute, or remove with justification.

Duplicates
Check exact and near repeats and key uniqueness. Confirm that repeated rows represent the same entity before deduplicating.

Invalid or inconsistent values
Check schema, types, labels, units, formats, and allowed ranges. Standardize valid variants; correct or reject invalid values.

Outliers
Flag unusual values with domain limits and robust summaries. Investigate each: an extreme may be valid, erroneous, or a measurement fault.

Cleaning is not automatic deletion: record the evidence, action, and rationale for each issue.

Data cleaning: assessments before treatment

Issue	Assessments to run	Decision to document
Missing values	Count and percentage by column/row; inspect patterns and co-occurrence; ask whether absence has domain meaning.	Preserve signal, impute, or remove; define the column-specific rule.
Duplicates	Check key uniqueness and exact/near matches; confirm the records describe the same entity or event.	Identify confirmed duplicates and document the removal rule.
Invalid or inconsistent	Check schema, types, labels, units, formats, and allowed ranges.	Define canonical values and correction or rejection rules.
Outliers	Check robust summaries and domain/time limits; investigate the source.	Classify as valid extreme or error; retain, correct, cap, transform, or remove with rationale.

Record and validate

Document the checks and decisions. Fit imputation within each training fold and apply it unchanged to held-out and production data.

Titanic / 2: cleaning decisions per column

Original figure not bundled

`images/class20_titanic_missingness.pdf`

Copy the original images/ folder here.

Titanic data used in `notebooks/Class4_DataEngineering.ipynb`, $n = 891$.

Column	Policy	Rationale
`deck`	explicit `Unknown`	77% missing: imputation would invent most values; missingness may be informative
`age`	median (28.0)	numerical, moderate missingness; the median is robust to outliers
`embarked`	mode (`S`)	two missing values: negligible effect
`embark_town`	explicit `Unknown`	duplicates `embarked` (see the χ^2 ranking)

The source analysis reports 107 duplicate-looking rows. Without a trusted identifier, equal feature values do not prove duplicated passengers: audit before dropping.

Feature construction from domain knowledge

- Informative features usually originate from an understanding of the problem rather than from generic mathematical transformations.
- Domain-informed counts, ratios, and rules can be more interpretable, robust, and sometimes more predictive than generic transformations.
- Each constructed feature should be checked for redundancy with the variables from which it was derived.
- The rationale for every constructed feature should be stated explicitly, in one sentence.

Titanic / 3: construct candidates from domain rules

Feature	Definition	Scale	Rationale
`family_size`	$\texttt{sibsp} + \texttt{parch} + 1$	ratio	Passengers travelling together share their fate; survival rate rises from 0.30 (alone) to 0.72 (four members), then falls
`is_alone`	$\mathbf{1}[\texttt{family_size} = 1]$	binary	Isolates the largest and least favoured group (537 passengers)
`title`	`man`→Mr, `woman`→Mrs, `child`→Master	nominal	One-to-one relabelling of `who`: adds no information (identical χ^2)

- From `notebooks/Class4_DataEngineering.ipynb`: each constructed feature is a short, explainable rule. A useful construction combines or transforms information; a bijective relabelling, such as `title`, only renames it.
- extbfAsk: Which transformation adds useful model capacity, and which merely renames an existing variable?

Titanic / 3: hypothesis before computation

Hypothesis

Survival does not necessarily change monotonically with the number of family members travelling together.

- `family_size = sibsp + parch + 1` aggregates related counts.
- `is_alone = 1[family_size = 1]` introduces a threshold response.
- `title` derived deterministically from `who` only relabels existing information in this teaching example.

Predict: Would `is_alone` help logistic regression more than a decision tree? Why?
We will revisit this hypothesis when we examine family-size survival rates.

Feature construction for streams and sensors

Original figure not bundled

`images/class20_stream_features.pdf`

Copy the original images/ folder here.

One batch generated by `pasteurization/synth_sensors.py`.

Features computed from the raw temperature T_t:

- **Rolling mean:** $\bar{T}_t = \frac{1}{w}\sum_{i=0}^{w-1} T_{t-i}$; smooths noise and indicates stability, at the cost of a lag of about $w/2$ samples.
- **Rate of change:** $\dot{T}_t = (T_t - T_{t-1})/\Delta t$; separates HeatUp ($> 0$) from Cool ($< 0$), which the raw value cannot.
- **Time in phase:** seconds since the last change of state; a state-aware feature that distinguishes a slow heating from a normal one.

All three use only past observations, so they can be computed online, one sample at a time, exactly as in production.

Represent candidates before selection

- Selection methods need features in a representation they can use: encode categorical variables and scale numeric ones when the method is scale-sensitive.
- Class 14 covered one-hot, ordinal, target, and cyclic encodings; the choice depends on the variable and the model.
- PCA transforms the feature space into components. It constructs a new representation rather than selecting original columns.
- Fit imputers, scalers, PCA, target-dependent selectors, and other learned transformations using training folds only; use out-of-fold target encoding.

Titanic / 4: representation and reproducible preprocessing

Numeric branch

`age`, `fare`, `family_size`
↓ fit median imputer on train
↓ fit scaler on train (if needed)

Categorical branch

`sex`, `embarked`
↓ handle missing category / mode
↓ fit one-hot encoder on train

Invariant

The validation/test and production data are *transformed* with the training-fitted objects: they are never used to estimate transformation parameters.

Question: What changes if the median is computed before cross-validation?

Feature selection: filter, wrapper, embedded

Given d candidate features, selection seeks a subset $S \subseteq \{1, \dots, d\}$ that maximizes a criterion $J(S)$. The three families differ in how J is defined.

Filter	J is a model-independent statistic computed for each feature, e.g. correlation, mutual information, or the χ^2 statistic.
Wrapper	$J(S)$ is the validation performance of a model trained on S; subsets are explored by a search strategy (e.g. forward or backward selection).
Embedded	Selection occurs during training, e.g. the least absolute shrinkage and selection operator (Lasso), $\min_\beta \|\mathbf{y} - X\beta\|_2^2 + \lambda\|\beta\|_1$, which sets some coefficients exactly to zero.

Titanic / 5: the χ^2 test of independence

Observed counts O (expected E under independence), `sex` $\times$ `survived`:

	died	survived	total
female	81 (193.5)	233 (120.5)	314
male	468 (355.5)	109 (221.5)	577
total	549	342	891

$$\chi^2 = \sum_{\text{cells}} \frac{(O - E)^2}{E} = 263.1$$

with $E = (\text{row total} \times \text{column total})/n$.

- If `sex` and `survived` were independent, each cell would contain approximately its expected count E.
- The larger χ^2, the further the observed table is from independence, i.e. the stronger the association with the target ($p < 10^{-58}$ here, with 1 degree of freedom).
- It is a filter criterion: computed per feature, before and independently of any model.
- **Caution:** `sklearn.feature_selection.chi2` treats feature values as non-negative counts. On label-encoded nominal variables it is not a contingency-table independence test; the score depends on arbitrary codes (92.7 versus 263.1 for `sex`).

Titanic / 5: a highly predictive feature may be invalid

Original figure not bundled

`images/class20_titanic_chi2.pdf`

Copy the original images/ folder here.

- **alive** attains the maximum $\chi^2 = n = 891$: it restates the target (`yes`/`no`). A filter ranks a leaking feature first; it must be removed by inspection, not by score.
- **embarked** and **embark_town** have identical scores: they encode the same information, so one of them is redundant.
- `who`, `adult_male`, and `sex` rank high together because they overlap strongly; a univariate filter cannot detect this redundancy.

χ^2 test of independence on the contingency table of each feature with `survived`; missing values treated as an explicit category.

Titanic / 6: is `is_alone` redundant?

Original figure not bundled

`images/class20_titanic_family_size.pdf`

Copy the original images/ folder here.

- **Informationally redundant:** `is_alone` is a deterministic function of `family_size`; it adds no information, which would justify removing it.
- **Not redundant for a linear model:** survival is non-monotonic in `family_size`, so its linear correlation with `survived` is only 0.02, whereas that of `is_alone` is -0.20. The indicator acts as a non-linear transformation that a linear model cannot build by itself.
- Redundancy is therefore relative to the model family: a decision tree can recover `is_alone` with a single split; logistic regression cannot. The notebook keeps it, and the test accuracy of its logistic regression rises from 0.804 to 0.810 (one additional correct prediction out of 179).

Titanic / 6: engineering changes feature dependencies

Original figure not bundled

`images/class20_titanic_correlation.pdf`

Copy the original images/ folder here.

- Pearson correlations after cleaning (age imputed with the median).
- `family_size` is strongly correlated with its components `sibsp` (0.89) and `parch` (0.78): keeping all three adds collinearity without new information.
- `is_alone` has a larger linear association with `survived` (-0.20) than `family_size` (0.02), as discussed in the selection example.
- Pearson correlation measures only linear dependence; a near-zero value does not imply that a feature is uninformative.

Titanic / 7: split first, then engineer within folds

Incorrect: validation information enters the pipeline

Whole data → imputation + selection → cross-validation → reported score

Correct: end-to-end pipeline fitted for each training fold

Split → [train-fold: impute → encode → select → fit model] → validate held-out fold

- Use stratification for this classification example; reserve an untouched test set for the final estimate.
- Supervised selection and hyperparameter tuning belong *inside* cross-validation (nested CV if estimating tuned-model generalization with CV).
- Deterministic row-wise rules are safe to predefine, but still check availability and deployment equivalence.

Titanic / 8: ablation evidence, not extra columns

Experiment	Difference vs. previous model	Test accuracy
A	Logistic regression without `is_alone`	0.804
B	Same notebook setup, add `is_alone`	0.810
C	Remove redundant candidate(s)	*To measure*
D	Alternative model family / selected subset	*To measure*

Interpretation

The existing notebook reports 0.804 $\rightarrow$ 0.810 (one additional correct prediction among 179 test examples). This is *not* enough to claim statistically reliable improvement.

Protocol for C/D: Same splits, fitted pipeline per fold, cross-validated scores and variation; evaluate the untouched test set once at the end.

Leakage and availability checks

- Is the feature available at prediction time, or does it use information from the future?
- Does it encode a post-decision outcome, such as the result of a manual review?
- Is it computed identically in production and in the notebook?
- Does it depend on joins or upstream systems that may be unavailable at inference time?

The complete taxonomy of leakage (target, preprocessing, temporal, group, duplicate, model selection) was presented in Class 13.

Titanic / 9: production-ready feature contract

Item	Deliverable / check
Data	Source and snapshot version, schema, missingness and range tests
Transformations	Serializable imputer, encoder, selector, and model (single pipeline)
Feature definitions	`family_size`, `is_alone`: exact formulas and types
Availability	Features exist at the prediction moment; `alive` excluded
Reproducibility	Code version, split seed, training config, metrics, artifact version
Monitoring	Missingness, category changes, input drift, and performance when labels arrive

Take-away: A notebook transformation becomes an engineering artifact only when it is reproducible, versioned, and executed consistently at inference time.

Repository connections

Repository activity

- `notebooks/Class4_DataEngineering.ipynb` covers cleaning, χ^2 selection, and domain-driven construction end to end.
- `pasteurization/synth_sensors.py` provides a contrastive online feature example (`dTdt`); rolling means must use past/current observations only.
- `streamlit/modeling.py` saves the fitted scaler together with the model, illustrating that preprocessing decisions must be persisted with the model rather than applied once in a notebook.

Papers and materials

- Guyon, I., Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Kohavi, R., John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Little, R. J. A., Rubin, D. B. (2019). *Statistical Analysis with Missing Data*, 3rd ed. Wiley.
- Gebru, T., Morgenstern, J., Vecchione, B., Wortman Vaughan, J., Wallach, H., Daumé III, H., Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
- Zheng, A., Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.
- scikit-learn User Guide: feature selection; pipelines and composite estimators.

Take-away message

Features are design decisions:

curate and validate data; construct with domain knowledge;
fit transformations and selection within training folds;
deploy only reproducible, available, monitored features.

Thank you for your attention.

Next: Class 17 — version control with Git.